

JavaScript Function Definitions

[Back to JS page](#)

Table of Contents

- [JavaScript Function Definitions](#)
 - [Function Declarations](#)
 - [Example 1](#)
 - [Function Expressions](#)
 - [Example 2](#)
 - [Hoisting](#)
 - [Example 3](#)
 - [The Function\(\) Constructor](#)
 - [Example 4](#)
 - [Functions are Objects](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

Function Declarations

A JavaScript function is defined with the `function` keyword, followed by a name, followed by parentheses `()`.

Function declarations are hoisted - they can be called before they are defined:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Declaration</h4>
<p id="demo"></p>

<script>
// Function declaration can be called before it's defined
let result = myFunction(5);
document.getElementById("demo").innerHTML = "Result: " + result;

function myFunction(x) {
  return x * x;
}
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Function Expressions

A function expression can be stored in a variable. The function can be anonymous (without a name):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Expression</h4>
<p id="demo"></p>

<script>
const x = function(a, b) {
  return a * b;
};
document.getElementById("demo").innerHTML = "4 * 3 = " + x(4, 3);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Hoisting

Hoisting is JavaScript's default behavior of moving declarations to the top of the current scope.

Function declarations are hoisted to the top. Function expressions are NOT hoisted:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hoisting</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Function declaration is hoisted
document.getElementById("demo1").innerHTML = "Declaration result: " + myDeclaration(5);

function myDeclaration(x) {
  return x * 2;
}

// Function expression is NOT hoisted
try {
  let result = myExpression(5);
  document.getElementById("demo2").innerHTML = "Expression result: " + result;
} catch(err) {
  document.getElementById("demo2").innerHTML = "Expression error: " + err.message;
}

const myExpression = function(x) {
  return x * 2;
};
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

The Function() Constructor

Functions can also be defined using the built-in JavaScript function constructor `new Function()` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Function() Constructor</h4>
<p id="demo"></p>

<script>
const myFunction = new Function("a", "b", "return a * b");
document.getElementById("demo").innerHTML = "4 * 3 = " + myFunction(4, 3);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Functions are Objects

The `typeof` operator returns "function" for functions. But functions can be treated as objects:

- Functions have properties (like `length`, `name`)
- Functions have methods (like `toString()`, `call()`, `apply()`)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Definitions</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Functions are Objects</h4>
<p id="demo1"></p>
<p id="demo2"></p>
<p id="demo3"></p>

<script>
function myFunction(a, b) {
  return arguments.length;
}

document.getElementById("demo1").innerHTML = "typeof: " + typeof myFunction;
document.getElementById("demo2").innerHTML = "Function length: " + myFunction.length;
document.getElementById("demo3").innerHTML = "Function toString: " + myFunction.toString();
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Function Definitions](#)